gems/faraday-0.8.8/lib/faraday.rb
```
module Faraday
  class << self
    def new(url = nil, options = {})
      # ...
      Faraday::Connection.new(url, options, &block)
    end

    # ...
```

The interesting stuff happens in Faraday::Connection#initialize. This method accepts an optional block and yields the newly created Connection object to the block:

gems/faraday-0.8.8/lib/faraday/connection.rb
```
module Faraday
  class Connection
    def initialize(url = nil, options = {})
      # ...
      yield self if block_given?
      # ...
    end

    # ...
```

This simple idiom is known as a *Self Yield*. Self Yields are pretty common in Ruby—even instance_eval and class_eval optionally yield self to the block, although this feature is rarely used in practice:

*Spell: Self Yield*

common_idioms/self_yield_in_eval.rb
```
String.class_eval do |klass|
  klass # => String
end
```

For a more creative example of a Self Yield, you can check out the tap method.

## The tap() Example

In Ruby, it's common to find long chains of method calls such as this:

common_idioms/tap.rb
```
['a', 'b', 'c'].push('d').shift.upcase.next # => "B"
```

Chains of calls are frowned upon in most languages (and sometimes referred to as "train wrecks"). Ruby's terse syntax makes call chains generally more readable, but they still present a problem: if you have an error somewhere along the chain, it can be difficult to track down the error.

For example, maybe you're worried that the call to shift is not returning what you expect. To confirm your suspicions, you break the chain and print out the result of shift (or set a breakpoint in your debugger):